

SGLANG-OMNI

Rethinking the Inference Engine for Multi-Stage Generative Models

SGLang Omni Team · github.com/sgl-project/sglang-omni

AGENDA

Four questions

01 **Multi-Stage Decoding**

What computation are we optimizing?

02 **Computational Characteristics**

What about it matters for serving?

03 **System Design**

What system follows from those characteristics?

04 **Progress and Roadmap**

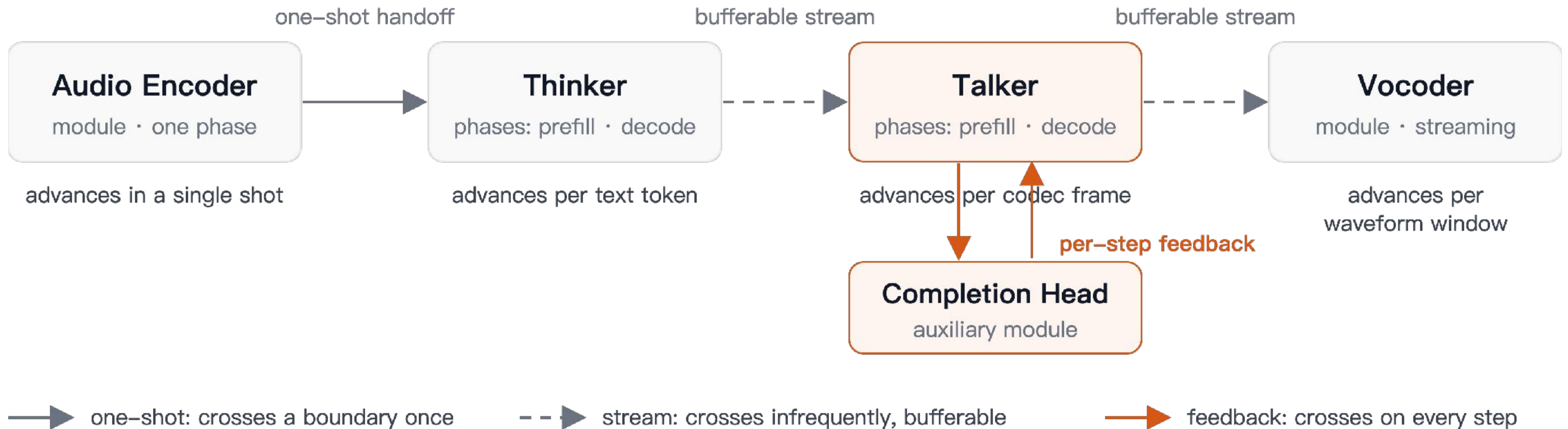
Where are we headed?

Part 01

Multi-Stage Decoding

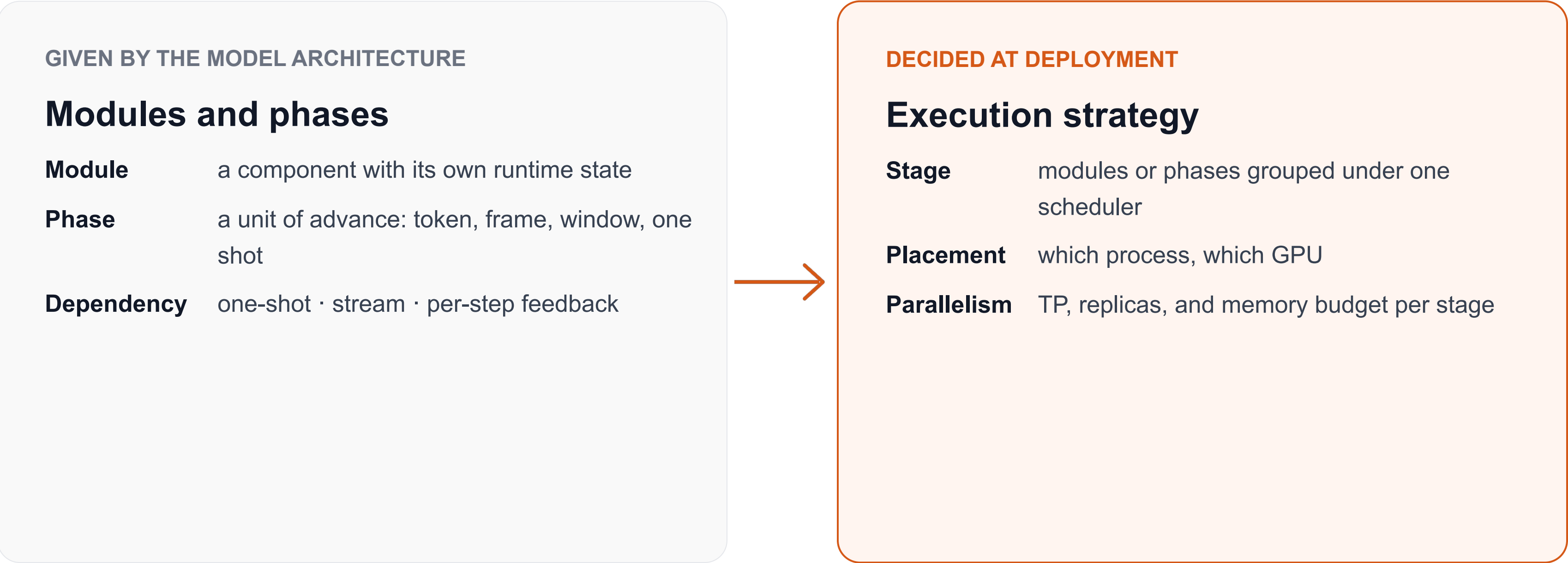
MULTI-STAGE DECODING

Generation beyond text traverses modules, phases, and dependencies



EXECUTION STRATEGY

The architecture declares modules; deployment must decide the strategy



Single-loop engines never had to make this decision. Multi-stage serving cannot avoid it.

THE REWARD

Two moves the architecture never mentions: fuse and replicate

Fuse



Per-step feedback stays inside the stage. No serialization, no synchronization, no transfer on every frame.

Replicate

PRELIMINARY

one GPU · three complete instances



<30%

SM activity, single instance

~2×

per-GPU throughput, three instances

Any strategy that preserves the data flow and the outputs realizes the same model.

Part 02

Computational Characteristics

THREE CHARACTERISTICS

Three measurable characteristics predict the right strategy

01 · dependency

Synchronization granularity

Read from the decoding algorithm

Decides the cost of a stage boundary. Feedback-coupled modules group into one stage.

02 · module

Bottleneck resource type

Measured on target hardware: counters + clock-scaling check

Decides the scheduling policy and whether idle capacity is worth reclaiming.

03 · module

Resident state size

Computed from the model: weights, KV bytes, activation peaks, buffers

Decides memory budgets and how many stages or instances fit on one device.

Host-bound + small resident state → several instances per GPU. Device-saturating → one.

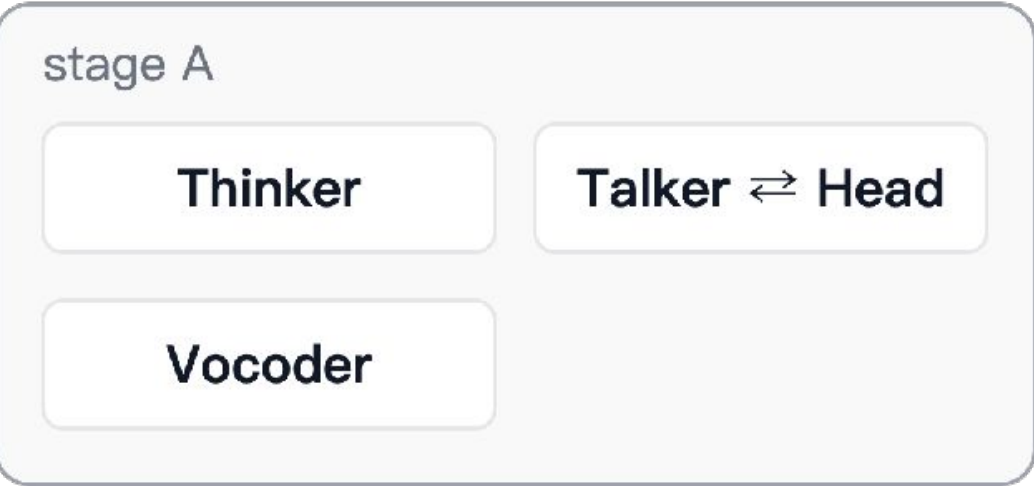
Part 03

System Design

The execution strategy becomes a configuration: stage → process → GPU

Fully fused

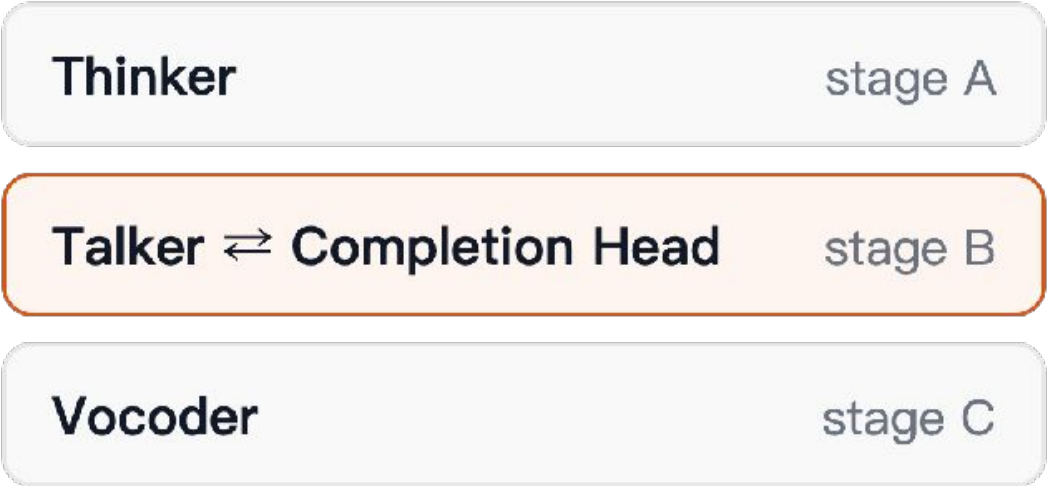
1 stage · 1 process · 1 GPU



boundaries per frame: 0
schedulers interfere;
thinker prefill delays first frames

Feedback-fused · default

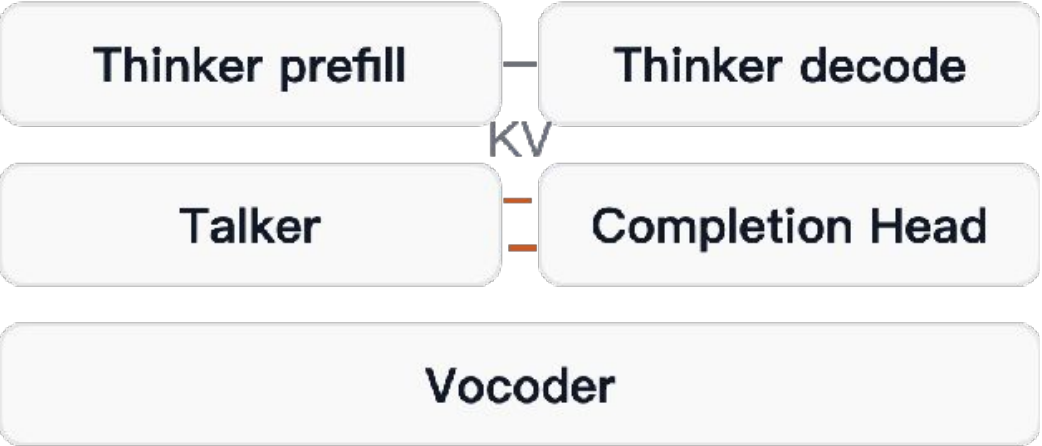
3 stages · 3+ processes · 1–2 GPUs



boundaries per frame: 0 (feedback stays in B)
streams cross A→B, B→C;
each stage its own scheduler

Disaggregated + PD

5 stages · 5+ processes · N GPUs

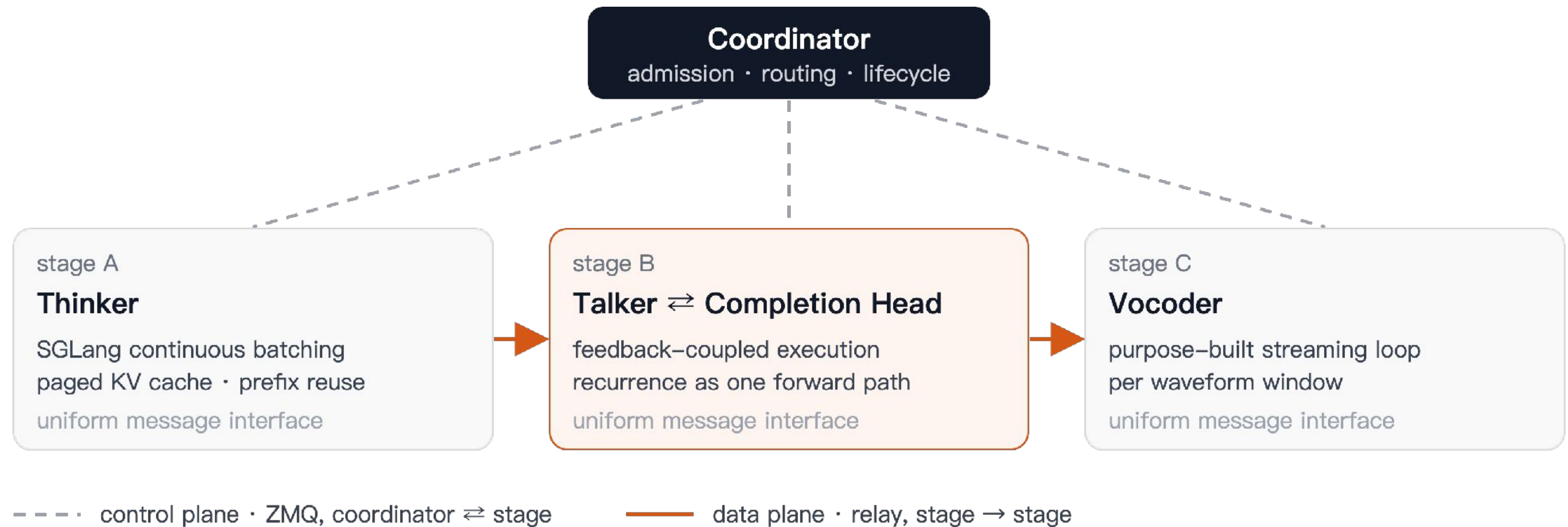


boundaries per frame: 1 (talker ⇌ head)
every dependency pays
sync + serialize + transfer

Same model, same outputs. PD disaggregation, fusion, and replication are all assignments of modules, phases, and instances to stages — one config change apart.

SCHEDULER DECOUPLING

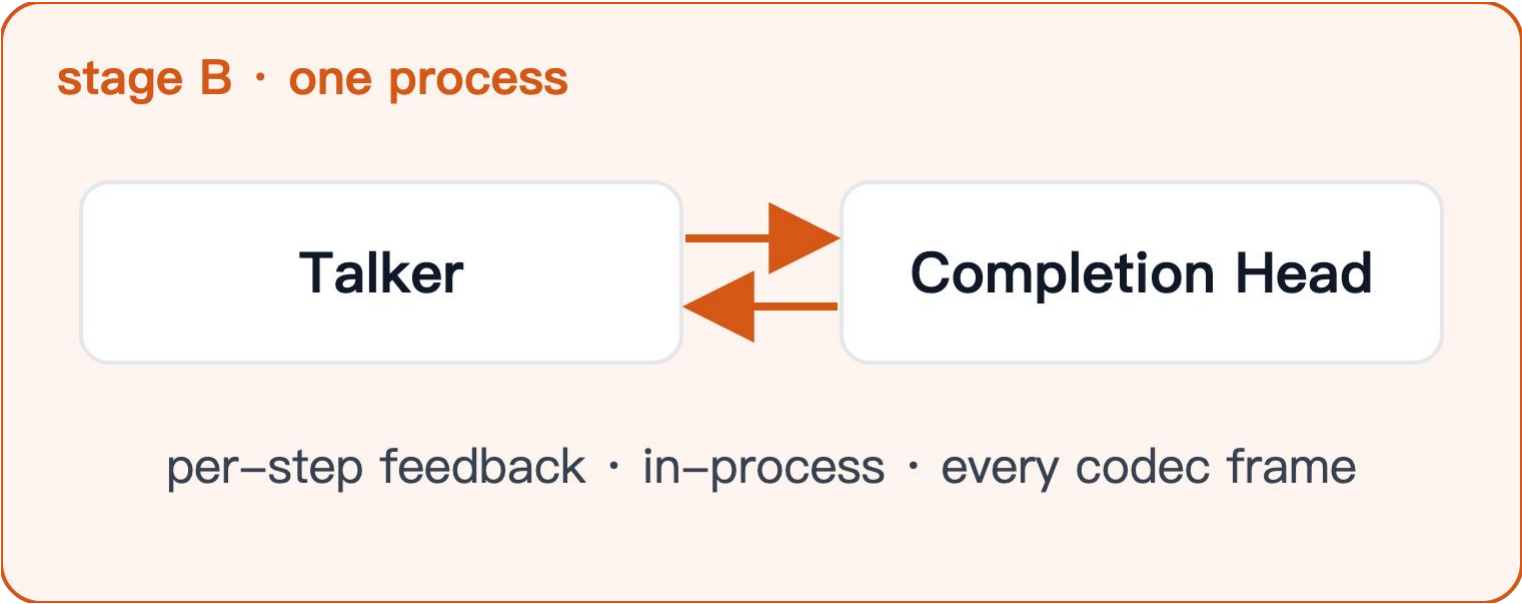
One scheduler per stage behind one uniform message interface



COMMUNICATION LAYERING

Control plane and data plane; transport follows placement

Placement of two stages	Data-plane transport
Same process	in-process handoff, tensor reference
Same GPU	device pointer via CUDA IPC, no copy
Same host	shared memory
Across devices	NVLink / RDMA interconnect

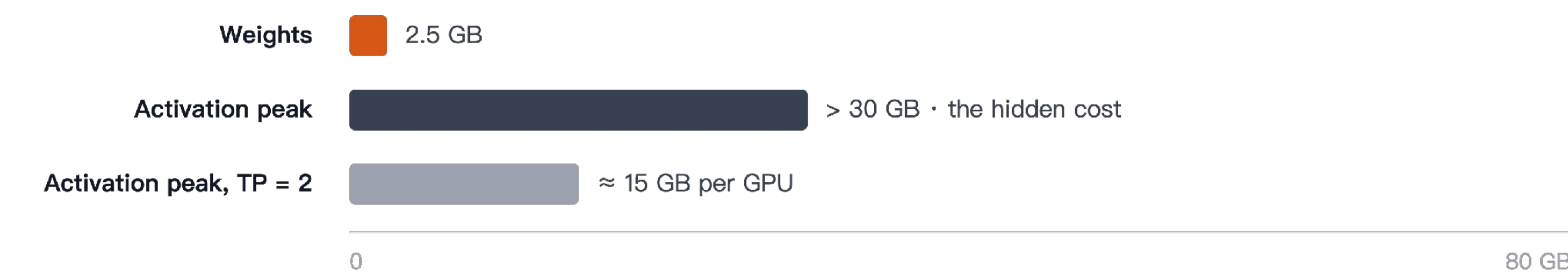


The tight loop never leaves the stage. Control messages stay on ZMQ; tensors never touch the control plane.

MEMORY ISOLATION

Per-stage memory budgets, declared and validated before serving

Qwen3–Omni audio encoder on one 80 GB GPU



Declared. Each stage states weights, KV pool, activation headroom, stream buffers.

Validated. The topology is checked against device memory before any process starts.

Isolated. A long encoder input cannot evict the thinker's KV cache.

Part 04

Progress and Roadmap

MODEL SUPPORT

Focused coverage: lead on voice, serve the frontier

TTS

performance and utility

Qwen3-TTS 1.7B · Base, CustomVoice
MOSS-TTS · Nano 100M
Fish S2 Pro
Fun-CosyVoice3 0.5B

ASR

streaming, long context, diarization

Whisper large-v3
Nemotron 3.5 ASR streaming 0.6B
MOSS-Transcribe-Diarize
Qwen3-ASR 1.7B

Omni and world

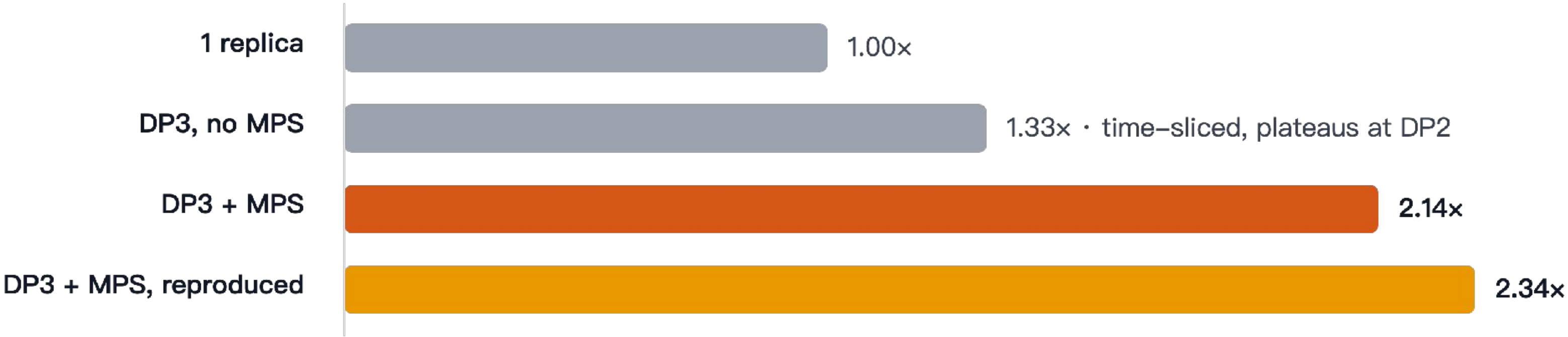
frontier multimodal inference

Qwen3-Omni 30B-A3B
NVIDIA Cosmos3-Super
Nemotron VoiceChat 11B
Ming-Omni · LLaDA2.0-Uni

Important and urgent first. RL, refactors, and hardware follow the same abstraction, so they compound rather than fork.

Replicas on one GPU recover the idle seventy percent

Relative throughput, one H100, TTS pipeline

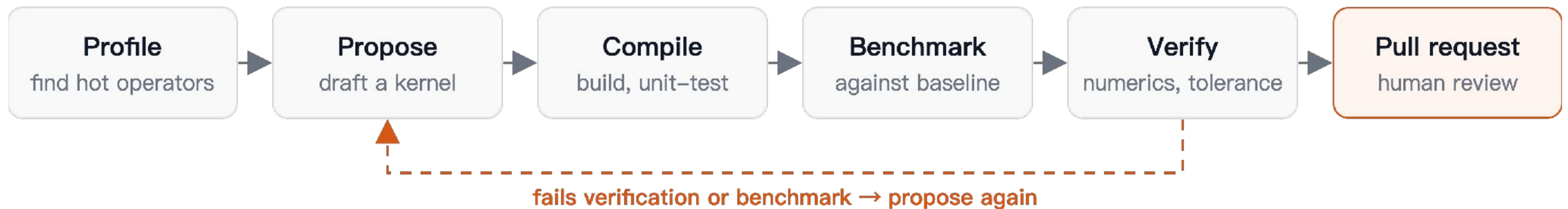


GPU idle 72% → ~30%. MPS lets replica kernels overlap instead of time-slice.

Ceiling is memory, not compute. Share one weight copy across replicas via CUDA IPC.

Router is on the hot path. Multi-process Python bridge now; Rust gateway as destination.

Agents that profile, write, verify, and benchmark kernels for new models

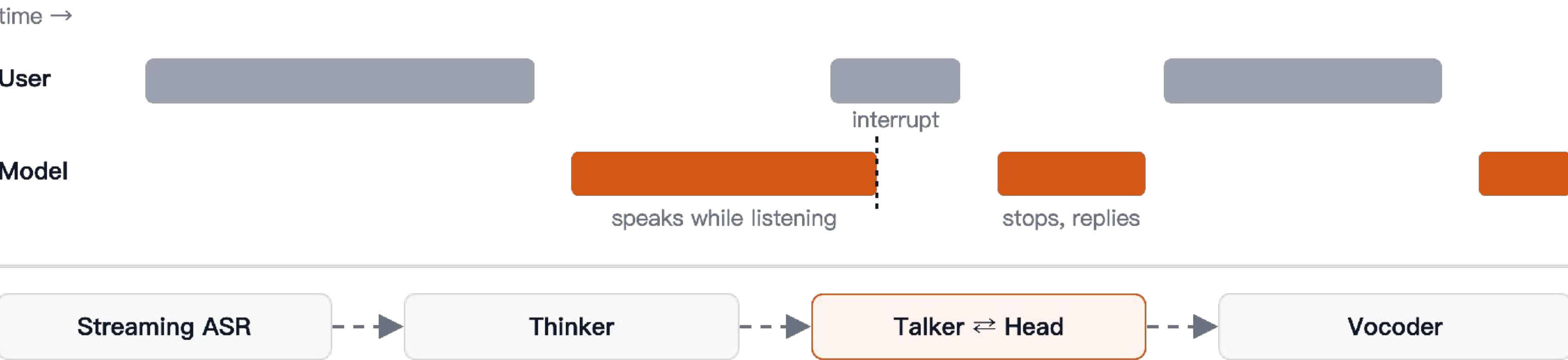


Reproduction targets: Qwen3-TTS and MOSS-Transcribe-Diarize — small enough to iterate, different enough to test generality.

First job of the agent: know when a kernel is the wrong fix. Host-bound stages need scheduler work, not faster kernels.

FULL DUPLEX

Listening and speaking in the same time window

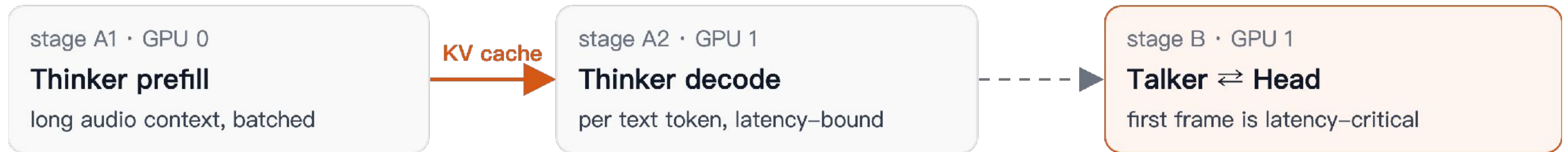


Input and output streams live together.
Real-time streams are first-class dependencies.

Interruption is preemption. Per-stage schedulers cancel in-flight frames without draining the thinker.

Continuity is the metric. Stall-free playback, not offline throughput.

Prefill and decode as two stages exchanging KV cache

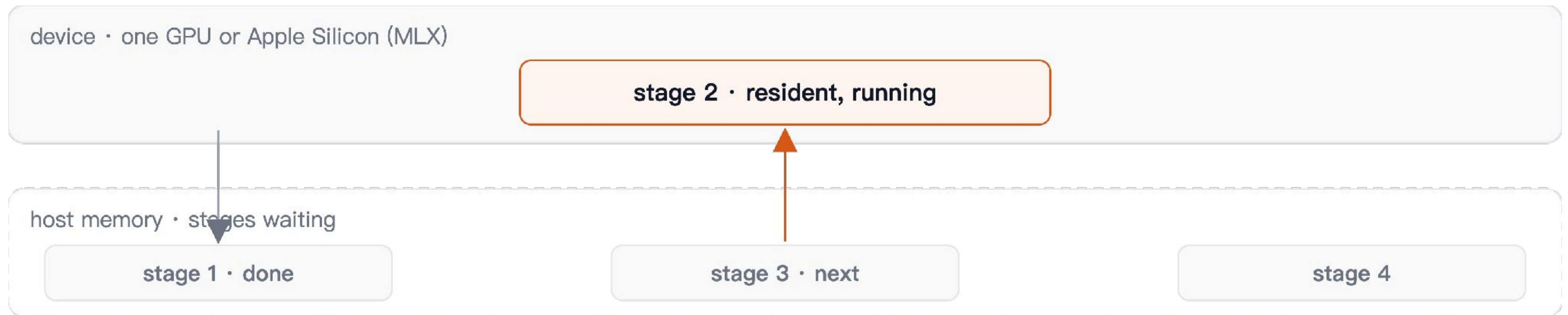


Already expressible. Two phases of one module, two stages; transport follows placement.

Roadmap makes it supported. KV transfer on the data plane; prefill scheduling that respects talker first-frame latency.

STAGE OFFLOAD

Request-serial stages let large pipelines fit small memory



MiniMax Music3. A long request-serial pipeline; only one stage needs to be resident at a time.

Same budgets, other direction. The topology time-shares one device instead of spreading across many.

Multi-stage rollouts for omni and TTS training

01

Per-stage weight updates

Broadcast new weights to the thinker without restarting the talker or vocoder.

02

Per-module outputs

Expose text, codec tokens, and audio for reward models that score any of them.

03

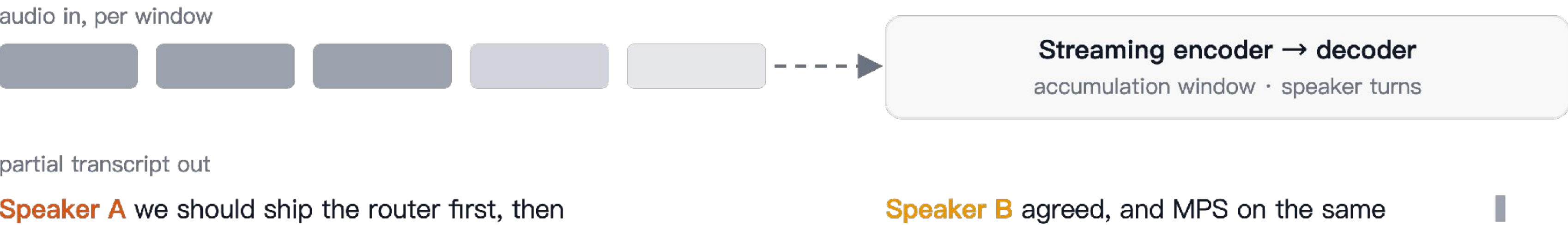
Stable topology

The same SPG configuration serves rollouts and production, so no train–serve skew.

RL frameworks assume a text-LLM rollout engine. Multi-stage models break that assumption the same way they broke serving.

ASR STREAMING

Real-time input, long context, multi-speaker



- Real-time input streaming. Chunked audio in, partial transcripts back, per window.
- Long context + diarization. MOSS-Transcribe-Diarize: hour-long meetings, speakers labeled, one card.
- Prerequisite for full duplex. The two tracks converge.

MULTI-HARDWARE

One runtime, one device abstraction, seven backends in flight

NVIDIA CUDA

reference · H100, H200,
consumer GPUs

Intel XPU · CPU

H2 2026 roadmap

AMD ROCm

tracking

Ascend NPU

Ming-Omni, Voxtral TTS landed

Moore Threads MUSA

RFC

Apple Silicon

MLX stage offload

Consumer GPUs

ASR and TTS serving

Device abstraction

a backend implements stage
loops, not the system

On weaker devices the GPU is no longer idle, so the best strategy changes with the hardware — one more reason it must be a configuration.

ROADMAP

Important and urgent first

Now

- Qwen3-TTS single-instance optimization
- Qwen3-Omni serving optimization
- Same-GPU DP + MPS recipes
- ASR streaming input
- Prefill CUDA graph
- Documentation and cookbook

Next

- Rust router, switchover from Python bridge
- PD disaggregation as a supported config
- KDA reproduction on two models
- RL utility for omni and TTS
- Stage offload: CUDA and MLX
- Hardware ports reach parity

Later

- Full duplex
- Multi-node inference and communication
- AutoRound quantization
- Model runtime profiling methodology
- Cosmos3 at scale
- Benchmark harness in CI

OPEN QUESTIONS

What we do not know yet

- 01 **Can the strategy be derived automatically?** The three characteristics predict it; we still write the config by hand.
- 02 **Mixed bottlenecks on one device.** A compute-bound DiT stage beside a host-bound AR stage: what scheduler?
- 03 **Streaming continuity as a benchmark.** How to measure stall-free playback fairly across systems?
- 04 **Rollouts with audio rewards.** The reward depends on audio; the audio depends on three stages.

Let's build together

`github.com/sgl-project/sglang-omni`

`slack.sglang.ai · #sglang-omni-dev`

